Intelligent AIOps Framework for Predictive CI/CD Pipeline Management and Autonomous Resource Optimization

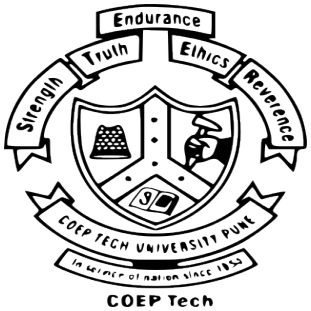
A Dissertation Report

Submitted by

Tejas Annasaheb Jagtap 712442021

Master of Technology in Information Security

Under the guidance of
Dr. V. K. Pachghare
COEP Technological University



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COEP TECHNOLOGICAL UNIVERSITY

June, 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING,
COEP TECHNOLOGICAL UNIVERSITY (COEP Tech) PUNE

CERTIFICATE

Certified that the dissertation titled, *"Intelligent AIOps Framework for Predictive CI/CD Pipeline Management and Autonomous Resource Optimization"* has been successfully completed by

Tejas Annasaheb Jagtap 712442021

and is approved for the partial fulfilment of the requirements for the degree of *"Master of Technology (CSE) – Information Security"*.

SIGNATURE

Dr. V. K. Pachghare
Project Guide
Dept. of Computer Science & Engineering
COEP Technological University,
Shivajinagar, Pune – 5.

SIGNATURE

Dr. P. P. Bartakke
Head of Department
Dept. of Computer Science & Engineering
COEP Technological University,
Shivajinagar, Pune – 5.

Contents

List of Figures iii

List of Tables iv

Abstract v

1 Introduction 1

1.1 Introduction to the Research Area 1

1.2 Motivation for Study 1

1.3 Research Gaps 2

1.4 Dissertation Problem Statement 3

1.5 Research Objectives 3

2 Literature Review 5

2.1 Detailed Literature Survey 5

2.2 Review Summary 8

3 Software Requirement Specification 10

3.1 Hardware Requirements 10

3.2 Software Requirements 10

3.3 Tools and Technology 11

4 Proposed High-Level System Design 13

4.1 Overview 13

4.2 System Architecture Components 13

4.3 System Architecture Diagrams 15

5 System Implementation 17

5.1 Synthetic Data Generation 17

5.2 Data Collection Infrastructure 17

5.3 Machine Learning Models 18

5.3.1 XGBoost Failure Prediction Model 18

5.3.2 TF–IDF + Autoencoder Log Anomaly Detection Model 18

5.3.3 LSTM CPU Forecasting Model 19

5.4 FastAPI AIOps Orchestrator . . . 19
5.5 Predictive Autoscaler . . . 20
5.6 Containerization and Kubernetes Deployment . . . 21
6 Results and Evaluation . . . 22
6.1 XGBoost Failure Prediction Results . . . 22
6.2 Anomaly Detection Results . . . 23
6.3 LSTM CPU Forecasting Results . . . 25
6.4 Combined Model Performance Summary . . . 26
6.5 Predictive Autoscaler Behaviour . . . 27
6.6 Summary of Implementation Outcomes . . . 28
7 Conclusion and Future Scope . . . 29
7.1 Conclusion . . . 29
7.2 Future Scope . . . 29
Bibliography . . . 31

List of Figures

4.1 Overall Overview of the Proposed AIOps System 15
4.2 System Architecture Showing Data Flow from CI/CD to ML Pipelines and Autoscaler 16

6.1 XGBoost Failure Prediction — Confusion Matrix 23
6.2 XGBoost Feature Importance Analysis 23
6.3 Log Anomaly Detection — Confusion Matrix 24
6.4 Reconstruction Error Distribution — Normal vs Anomalous Logs 25
6.5 LSTM CPU Forecasting — Actual vs Predicted Values 26
6.6 LSTM CPU Forecasting — Actual vs Predicted Scatter Plot 26
6.7 Model Comparison — Classification Metrics and Forecasting Metrics 27

List of Tables

2.1 Detailed Literature Survey 5

5.1 AIOps Orchestrator — REST API Endpoints 20
5.2 Predictive Autoscaler — Decision Policy (Algorithm 1) 20

6.1 XGBoost Failure Prediction — Test Set Performance 22
6.2 XGBoost 5-Fold Cross-Validation Results 22
6.3 Log Anomaly Detection — Test Set Performance 24
6.4 LSTM CPU Forecasting — Test Set Performance 25
6.5 AIOps System — Combined Model Performance Summary 27
6.6 Predictive Autoscaler — Decisions for Representative Scenarios 27

Abstract

Modern software development pipelines rely heavily on Continuous Integration and Continuous Deployment (CI/CD) systems to ensure rapid and reliable delivery. As organizations scale, these pipelines generate massive volumes of logs and highly variable workloads, making it increasingly difficult to manually detect failures, understand anomalies, and maintain optimal resource usage. Traditional monitoring tools and Kubernetes' reactive autoscaling mechanisms are often insufficient in anticipating failures or predicting future resource demands. This leads to delays, inefficient resource allocation, and increased operational overhead.

This dissertation presents the design, complete implementation, and evaluation of an intelligent AIOps-based framework that integrates machine learning into CI/CD and cloud-native infrastructure operations. The system delivers three core capabilities: (1) predicting build failures in advance using an XGBoost classifier trained on Jenkins metadata, (2) detecting anomalies in CI log streams using a TF–IDF based Autoencoder model implemented in PyTorch, and (3) forecasting future CPU usage in Kubernetes clusters through an LSTM-based time-series prediction model. These models are unified under a FastAPI-based AIOps orchestrator that loads all models at startup and serves real-time inference through REST APIs. A custom predictive autoscaler consumes these inference outputs to make proactive Kubernetes scaling decisions, replacing the reactive threshold-based behaviour of the default Horizontal Pod Autoscaler.

The complete system has been implemented and validated end-to-end. On held-out test data, the XGBoost failure-prediction model achieved 99.17% accuracy and a 98.18% F1-score, the TF–IDF Autoencoder anomaly detector reached 96.67% accuracy with a 93.10% F1-score, and the LSTM CPU forecaster attained an RMSE of 0.070 and a MAPE of 9.68%. All three models were integrated into the orchestrator, the predictive autoscaler was implemented using the Kubernetes Python client, and the entire pipeline — from data ingestion through model inference to autoscaling decisions — was packaged with Docker and Kubernetes manifests. The results demonstrate the potential of AIOps to transform DevOps from reactive processes into proactive, data-driven automation.

Chapter 1

Introduction

1.1 Introduction to the Research Area

Over the last decade, software development practices have undergone a major shift with the rise of DevOps, cloud-native infrastructure, and continuous automation. Modern development teams no longer depend on slow, manually executed build and deployment cycles. Instead, Continuous Integration and Continuous Deployment (CI/CD) pipelines have become the backbone of software delivery. These pipelines automatically compile code, execute tests, and deploy applications to production environments. As organizations adopt microservices, containerization, and distributed systems, CI/CD now plays an essential role in maintaining fast release cycles and ensuring product stability.

At the same time, Kubernetes has emerged as the de-facto platform for large-scale container orchestration. It manages application deployment, scaling, and resource allocation in cloud-native environments. While Kubernetes is powerful, its default automation mechanisms especially the Horizontal Pod Autoscaler (HPA) rely only on reactive rules. They respond to resource spikes after they occur rather than anticipating them.

This transformation in development and deployment workflows has created a large ecosystem of operational data. CI/CD tools like Jenkins generate thousands of log lines per build, containing warnings, test outputs, exceptions, and detailed execution traces. Prometheus captures resource usage metrics from Kubernetes clusters continuously. Developers and DevOps engineers must interpret this data to keep systems healthy, but doing so manually is increasingly impractical.

This is where the field of AIOps (Artificial Intelligence for IT Operations) has gained importance. AIOps integrates machine learning, data analytics, and automation to enhance operational decision-making. It offers capabilities like anomaly detection, real-time insight extraction, predictive analysis, and automated remediation. The research area explored in this dissertation lies at the intersection of AIOps, CI/CD intelligence, and cloud-native resource management. The core idea is to move DevOps from a reactive model to a proactive and self-learning one.

1.2 Motivation for Study

My motivation for choosing this topic comes from observing the real challenges faced by development teams working with modern CI/CD pipelines. Even with automation tools, pipeline failures often appear unexpectedly. A single unstable dependency, hidden error, or misconfigured step can cause a build to fail after running for several minutes. This wastes

developer time and slows down release cycles. Many times, the root causes are buried deep within lengthy logs, making manual troubleshooting cumbersome.

Similarly, Kubernetes clusters handle highly dynamic and unpredictable workloads. Traditional autoscaling reacts to CPU or memory spikes only after the threshold is breached. This delay, even if just a few seconds, can cause service degradation, latency, timeouts, or pod failures. When teams rely heavily on microservices, these issues become even more pronounced.

From a DevOps engineer's perspective, the increasing volume of logs, metrics, and build data has become too large to analyze manually. Even experienced engineers find it difficult to track anomalies hidden within logs or predict failure-prone builds in advance. I felt that there was a strong need for an intelligent system that learns from historical patterns, continuously monitors system behavior, and takes decisions automatically.

The motivation for this dissertation, therefore, is to design and evaluate a system that integrates machine learning into CI/CD and Kubernetes operations. By analyzing pipeline history, logs, and performance data, the system aims to provide early insights that would otherwise require experienced humans to detect. The proposed solution not only benefits DevOps engineers but also contributes to higher software quality, reduced downtime, and better utilization of cloud infrastructure.

1.3 Research Gaps

Although CI/CD automation tools and Kubernetes scaling mechanisms are widely used, they still operate with significant limitations. A review of existing literature and industry practices reveals several research gaps that justify the need for this dissertation.

First, most CI/CD systems lack the ability to predict failures. They can identify problems only after they occur, often after long build processes that consume time and compute resources. The absence of predictive intelligence means that many failures especially those caused by recurring patterns or subtle anomalies could have been prevented if early warning signals were identified.

Second, log analysis within CI/CD systems is still largely reactive and manual. Logs contain rich signals about failures and unusual behavior, but due to their unstructured nature and large volume, humans cannot realistically monitor them in real time. Existing systems do not use advanced machine learning techniques like Autoencoders or Transformer-based models to detect anomalies as they occur.

Third, Kubernetes autoscaling is fundamentally reactive. Tools such as HPA depend on CPU or memory readings crossing predefined thresholds. These metrics provide no insight into upcoming workload spikes, such as developers planning to run multiple test jobs or a sudden increase in CI activity. Many studies focus either on autoscaling or log anomaly detection, but very few attempt to integrate these two domains an area that this dissertation aims to explore.

Lastly, existing research typically treats CI/CD failure prediction, log anomaly detection,

and autoscaling as isolated problems. In practice, these issues are deeply interconnected. A failure in a pipeline may be caused by anomalies in logs; high workload may cause resource exhaustion; and insufficient scaling may lead to build timeouts. However, most solutions do not combine these signals into a single intelligent framework capable of responding holistically.

These gaps provide a strong justification for the integrated AIOps-driven system proposed in this dissertation.

1.4 Dissertation Problem Statement

Based on the identified research gaps, the core problem addressed in this dissertation can be stated clearly as follows:

"To design and implement an intelligent AIOps-based framework that can proactively predict CI/CD pipeline failures, detect anomalies in Jenkins logs in real time, and autonomously optimize Kubernetes resources through predictive autoscaling, thereby improving pipeline reliability, reducing operational delays, and minimizing infrastructure costs."

This problem focuses on replacing reactive DevOps mechanisms with proactive, data-driven decision support powered by machine learning.

1.5 Research Objectives

To address the problem statement effectively, the following research objectives were defined and have been accomplished in this dissertation:

1. **Develop a Data Collection Framework for CI/CD and Cluster Metrics:** Design and implement automated data collectors that extract structured build metadata, console logs, and CPU usage metrics from Jenkins and Prometheus, and a synthetic data generator to produce training-ready datasets that mirror production CI/CD environments.
2. **Build Machine Learning Models for Failure Prediction, Anomaly Detection, and CPU Forecasting:** Train and evaluate three ML components — an XGBoost classifier for predicting CI build failures before execution, a TF–IDF based Autoencoder model for detecting anomalous log patterns, and an LSTM neural network for forecasting CPU usage trends in Kubernetes workloads.
3. **Implement a FastAPI-Based AIOps Orchestrator for Intelligent Decision-Making:** Develop a centralized inference layer that integrates all ML models, loads them at startup, and exposes RESTful endpoints that generate actionable insights such as predicted failures, anomaly alerts, and CPU forecasts in real time.
4. **Design a Predictive Autoscaling Engine for Kubernetes Using ML Outputs:** Create a Kubernetes-compatible autoscaler that uses ML predictions — failure probability, anomaly scores, and CPU forecasts — to proactively scale workloads, reduce

resource wastage, and enhance CI/CD reliability, and validate the complete pipeline end-to-end.

Chapter 2

Literature Review

2.1 Detailed Literature Survey

Table 2.1: Detailed Literature Survey

Paper Name	**Author(s)**	**Work Done**	**Gaps / Takeaway (Relevance to Project)**
Enhancing Machine-Learning-Based Autoscaling for Cloud Resource Orchestration	I. Pintye et al. (2024)	Evaluates ML models for autoscaling cloud applications; compares feature sets and model architectures for predicting load and controlling scale actions.	Focuses on autoscaling alone; lacks integration with CI/CD signals or log anomaly inputs. Does not address end-to-end AIOps orchestration across prediction, anomaly detection and scaling.
Time-series Forecasting-Based Kubernetes Autoscaling using Facebook Prophet and LSTM	P. B. Guruge & Y. H. P. P. Priyadarshana (2025)	Proposes a hybrid forecasting approach (statistical + LSTM) to predict cluster metrics and drive scaling decisions; evaluates forecast horizons and autoscale rules.	Good forecasting results, but work evaluates autoscaling in isolation and does not incorporate CI pipeline or log features (failure predictions/anomalies). Limited discussion on real-time log integration.

Continued on next page

Table 2.1 – continued from previous page

Paper Name	**Author(s)**	**Work Done**	**Gaps / Takeaway (Relevance to Project)**
Deep Learning for Multivariate Time-Series Anomaly Detection: A Survey	M. A. Yahya et al. (2025)	Comprehensive survey of reconstruction-based anomaly detectors (autoencoders, VAEs, PCA, etc.) for multivariate time series; benchmarks methods and discusses thresholds.	Survey-level work; does not present an integrated DevOps pipeline case. Guidance on thresholding useful, but lacks concrete application to text-based CI logs (TF-IDF → Autoencoder pipeline).
Towards Optimization of Anomaly Detection in DevOps	A. Hrusto (2023)	Applies deep learning techniques for detecting anomalies in monitoring and log data in DevOps contexts; experiments with representation learning and feature selection.	Focuses mainly on monitoring metrics rather than long textual console logs; limited exploration of the combination of supervised failure prediction and unsupervised log anomaly detection.
AI-Enhanced Platform Engineering: Revolutionizing CI/CD Pipelines Through Intelligent Automation	Research overview (2025)	High-level study describing application of AI across CI/CD: predictive alerts, log triaging, and smart orchestration; includes case studies and proposed architecture patterns.	Conceptual and high level; lacks detailed experiments, concrete model evaluation, or implementation of a closed-loop autoscaler integrating failure predictions and log anomalies.

Continued on next page

Table 2.1 – continued from previous page

Paper Name	**Author(s)**	**Work Done**	**Gaps / Takeaway (Relevance to Project)**
Kubernetes Autoscaling with Machine Learning Based on Traffic Load Prediction	Y. M. Abdelhamid, W. A. Aziz, J. N. Soliman (2025)	Demonstrates ML-based autoscaling driven by traffic forecasts; evaluates different ML models for traffic prediction and scaling rules on Kubernetes.	Mainly traffic-centric; does not incorporate CI/CD-specific indicators such as build queue length or log anomaly signals that are important for pipeline-aware autoscaling.
Predictive Failure Analysis in CI Pipelines using Ensemble Trees	R. Mathew & S. Baswareddy (2025)	Uses ensemble tree models (including gradient boosting) to predict CI build failures from metadata and limited log-derived features; presents feature importance analysis.	Good match for failure prediction, but dataset size and diversity are limited; work does not combine anomaly detection or predictive scaling into a single framework.
LogBERT: Transformer-Based Log Anomaly Detection for CI/CD Systems	H. Guo et al. (2021)	Introduces Transformer (BERT-style) models for log representation and anomaly detection; shows improved detection of sequence-level anomalies compared with TF-IDF + Autoencoder baselines.	High accuracy but computationally heavy; needs large labeled or pretraining corpora. Less suitable for lightweight, near-real-time deployment on small lab setups without GPUs.

Continued on next page

Table 2.1 – continued from previous page

Paper Name	Author(s)	Work Done	Gaps / Takeaway (Relevance to Project)
Auto-Scaling Techniques for Container Workloads in Kubernetes Clusters — A Review	Various Authors (2025)	Survey of HPA/VPA, custom autoscalers, rule-based vs ML-based approaches; discusses pros/cons, reactive vs predictive strategies, and common metrics used.	Useful background material; survey highlights the need for richer signals (logs, CI metrics) but does not propose an integrated AIOps loop or evaluate combined signals.
Empirical Analysis on CI/CD Pipeline Evolution: Failures and Bottlenecks	S. Houerbi et al. (2024)	Empirical study of real-world CI/CD pipelines, identifying common failure causes, temporal patterns, and resource bottlenecks across projects.	Descriptive study identifies problems but does not propose automated ML solutions. Useful for motivating features but lacks prescriptive integrated remediation strategies.

2.2 Review Summary

The literature survey shows that existing research addresses these challenges individually, but rarely in an integrated manner. For example, Pintye et al. (2024) highlight how ML-based autoscalers outperform traditional threshold-driven Kubernetes HPA, but their method relies solely on resource metrics and ignores pipeline behaviour that often drives workload changes. Similarly, Guruge and Priyadarshana (2025) use Prophet and LSTM for CPU forecasting, showing promising accuracy, but their approach does not incorporate CI/CD signals or log anomalies, restricting its ability to act proactively in real-world DevOps pipelines.

In the domain of anomaly detection, Yahya et al. (2025) provide a comprehensive survey of reconstruction-based deep learning models for multivariate anomalies. Their findings confirm the effectiveness of Autoencoders for identifying deviations, yet their work focuses mainly on structured system metrics rather than unstructured CI/CD logs. Meanwhile, Guo et al.'s LogBERT (2021) shows how transformer architectures can learn contextual log patterns more effectively than traditional models, though it demands large datasets and computation, making it less suitable for lightweight operational integration.

On the CI/CD reliability side, Mathew and Baswareddy (2025) analyse build history and use ensemble models such as Gradient Boosting and XGBoost to predict failures with strong ac-

curacy. However, the study treats CI/CD failures as isolated events rather than linking them with anomalies or infrastructure behaviour. Houerbi et al. (2024) further emphasize that many pipeline failures arise from hidden log patterns, configuration drift, and environmental inconsistencies signals that cannot be captured by metadata alone.

Across all reviewed works, the dominant research gap is that no existing system combines failure prediction, log anomaly detection, and predictive autoscaling into a unified AIOps framework. Existing solutions focus on singular parts resource metrics, logs, or build metadata leading to partial visibility. A holistic system integrating these signals could provide deeper situational awareness, more reliable pipelines, and proactive infrastructure optimisation. This gap forms the foundation of the dissertation's proposed architecture.

Chapter 3

Software Requirement Specification

3.1 Hardware Requirements

The hardware requirements for the system are primarily driven by the computational needs of training machine learning models and running the AIOps orchestrator. The following configurations are recommended:

Minimum Hardware Requirements

- **Processor:** Intel i5 (8th generation or higher) / AMD equivalent
- **Memory (RAM):** 8 GB
- **Storage:** 50 GB of free disk space
- **GPU:** Not mandatory; the models train on CPU, though a GPU accelerates Autoencoder and LSTM training

Recommended Hardware Requirements

- **Processor:** Intel i7/i9, AMD Ryzen 7 or above
- **Memory (RAM):** 16–32 GB, to support parallel feature extraction and model training
- **Storage:** 100–250 GB SSD (for datasets, model checkpoints, logs)
- **GPU:** NVIDIA CUDA-enabled GPU (4–8 GB VRAM) for faster training of the Autoencoder and LSTM

3.2 Software Requirements

The software stack consists of operating system support, programming languages, libraries, and containerization environments required to implement the AIOps framework.

Operating System

- Windows 10/11 (for development and experimentation)
- Linux-based system (Ubuntu 20.04 or later) for running Kubernetes clusters

Programming Languages and Frameworks

- **Python 3.10+**: Core implementation language for ML models, collectors, and orchestrator.
- **FastAPI**: Framework for building the AIOps Orchestrator REST API.
- **PyTorch**: For implementing and training the Autoencoder and LSTM deep-learning models.
- **XGBoost**: For build failure prediction.
- **Scikit-learn**: For preprocessing, TF–IDF vectorization, train/test splitting, and evaluation metrics.

Libraries and Dependencies

- NumPy, Pandas (data processing)
- Matplotlib (evaluation plots and visualization)
- Joblib (serialization of the XGBoost model, TF–IDF vectorizer, scaler, and threshold)
- Pydantic (FastAPI request/response models)
- Uvicorn (ASGI server)
- Kubernetes Python client (autoscaler scaling actions)
- Requests / python-jenkins (Jenkins and Prometheus HTTP access)
- python-dotenv (environment variables)

External Services

- **Jenkins**: Source of CI/CD logs and build metadata
- **Prometheus**: Source of CPU and system metrics
- **Kubernetes API Server**: Used by the autoscaler for applying scaling actions

3.3 Tools and Technology

The development and deployment of the AIOps framework require a combination of DevOps, ML, and containerization tools. The following tools and technologies are used:

- **Visual Studio Code**: Primary IDE for development.
- **Docker**: Containerization of the orchestrator and autoscaler components.
- **Minikube / Kubernetes Cluster**: Deployment and testing of the predictive autoscaler.
- **Prometheus & Grafana**: Metric collection and monitoring dashboards.

- **GitHub**: Version control and collaborative code management.
- **Jenkins**: CI/CD tool providing logs and build metadata.
- **Python Kubernetes Client**: For implementing the custom autoscaling logic.
- **Postman / Curl**: For testing the FastAPI endpoints during development.

Together, these hardware and software components form a robust environment necessary to develop, train, deploy, and evaluate the intelligent AIOps framework presented in this dissertation.

Chapter 4

Proposed High-Level System Design

4.1 Overview

The proposed AIOps-based framework integrates multiple intelligent components that work together to enhance the reliability, efficiency, and scalability of CI/CD pipelines running in cloud-native environments. Modern DevOps ecosystems consist of several interacting elements such as CI tools, log streams, monitoring systems, and Kubernetes-based deployments. Traditional approaches treat these components independently, but the increasing complexity and scale of software delivery demand a unified design that allows these systems to exchange information, learn from historical behavior, and make proactive decisions.

The high-level design of the system centers around three primary machine learning modules — failure prediction, log anomaly detection, and CPU workload forecasting — and a FastAPI-based orchestration layer that connects them to external systems like Jenkins, Prometheus, and Kubernetes. Rather than relying on reactive or threshold-based automation, the architecture enables the system to anticipate failures, detect anomalies in real-time, and adjust compute resources proactively. This chapter describes the major components of the architecture, their interactions, and the overall system workflow.

4.2 System Architecture Components

The architecture is composed of several interconnected modules, each contributing to the system's ability to perform predictive and autonomous operations.

1. Jenkins Data Collection Layer

Jenkins serves as the primary source of CI/CD pipeline information. This component retrieves build metadata such as build duration, timestamps, status labels, and test results, along with detailed console logs that capture every step in the build process. The Jenkins REST API is used to extract these records, which are then preprocessed for downstream machine learning tasks. Console logs, being unstructured, are cleaned and normalized before vectorization using TF-IDF.

2. Prometheus Metrics Collector

Prometheus acts as the monitoring backend of the architecture. It collects time-series metrics from the Kubernetes cluster, including CPU utilization, memory usage, and pod replica counts. The collected data is essential for training and running the LSTM-based CPU fore-

casting model. The metrics are periodically fetched through the Prometheus HTTP API and stored in structured formats suitable for training.

3. Machine Learning Models

The framework integrates three ML models:

- **Failure Prediction Model (XGBoost):** Trained on structured Jenkins metadata, this model predicts whether a build is likely to fail before execution. It helps reduce wasted compute time and enhances CI/CD reliability.
- **Log Anomaly Detection Model (Autoencoder):** Using TF-IDF vectors derived from console logs, the Autoencoder learns patterns of normal behavior. Deviations generate high reconstruction errors, allowing real-time identification of unusual or potentially hazardous log segments.
- **CPU Forecasting Model (LSTM):** Built using time-series metrics from Prometheus, this model forecasts CPU usage several minutes ahead. Its predictions enable proactive scaling decisions.

4. FastAPI-based AIOps Orchestrator

The orchestrator acts as the central communication hub. It loads the trained ML models into memory and exposes a set of RESTful APIs that other components can interact with. These include:

- `/predict_failure`: returns failure probability
- `/detect_anomaly`: returns anomaly score and flag
- `/forecast_cpu`: provides CPU forecasts
- `/autoscale`: triggers the predictive autoscaling engine

The orchestrator ensures modularity, allowing models to be updated independently without altering pipeline logic.

5. Predictive Autoscaler

Traditional Kubernetes Horizontal Pod Autoscaler (HPA) performs scaling based purely on current CPU metrics. In contrast, the predictive autoscaler uses LSTM forecasts to scale resources ahead of time. It also incorporates anomaly scores and failure probabilities to avoid unnecessary scaling during unstable conditions. The autoscaler communicates with the Kubernetes API server to update replica counts and enforces constraints such as cooldown intervals, minimum/maximum replicas, and rate limits.

4.3 System Architecture Diagrams

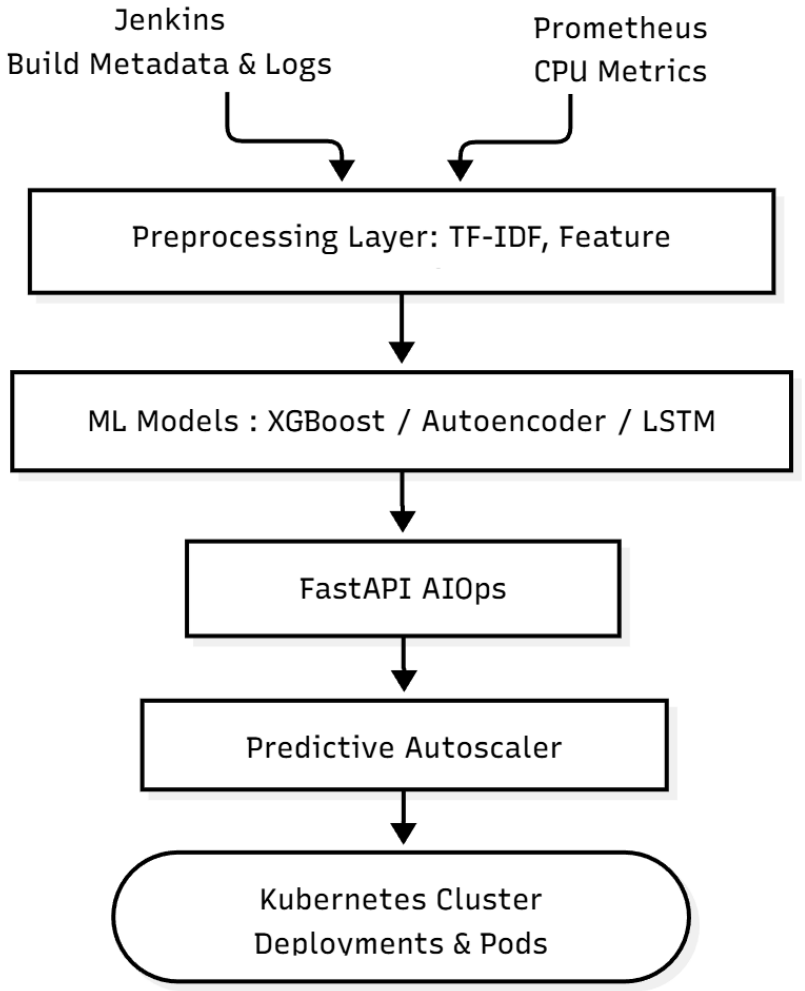


Figure 4.1: Overall Overview of the Proposed AIOps System

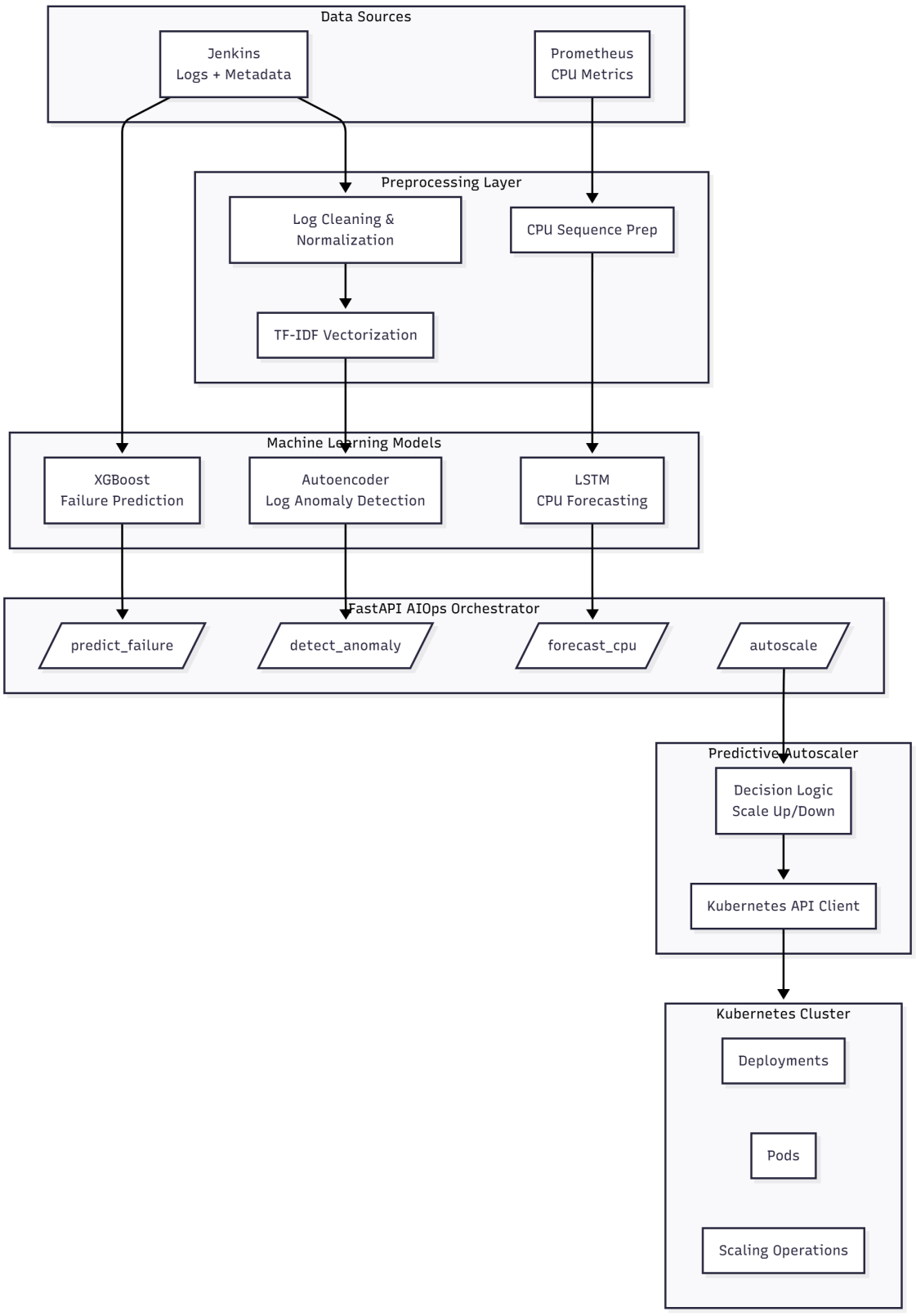


Figure 4.2: System Architecture Showing Data Flow from CI/CD to ML Pipelines and Autoscaler

Chapter 5

System Implementation

This chapter presents the complete implementation of the AIOps framework. The work spans the entire pipeline: synthetic data generation, data collection infrastructure, training of all three machine learning models, the FastAPI orchestration layer that serves them, the predictive autoscaler that consumes their outputs, and the containerized Kubernetes deployment. The implementation was carried out on a Windows environment using Python 3.10+, with PyTorch for the deep-learning models (Autoencoder and LSTM) and XGBoost for the classification task. All trained artifacts are serialized into `orchestrator/models/` so that the orchestrator can load them at startup.

5.1 Synthetic Data Generation

Since real-world CI/CD data from Jenkins and Prometheus was limited in volume, a comprehensive synthetic data generation module was developed to produce training-ready datasets that closely mimic production CI/CD environments. The data generator produces three categories of synthetic data:

- **Jenkins Build Metadata:** 600 synthetic build records were generated with realistic distributions across SUCCESS (∼68%), FAILURE (∼23%), and UNSTABLE (∼9%) outcomes. Each record includes features such as duration, number of warnings, errors, test results (failed, passed, total), and log length. Feature correlations were carefully designed — for example, failed builds tend to have longer durations, more errors, and more test failures.
- **Jenkins Console Logs:** 600 synthetic log files were generated using realistic templates that replicate actual Jenkins pipeline output. Logs include stages such as checkout, dependency installation, linting, testing, and build steps. Failure and success logs have distinct vocabulary patterns, making them suitable for anomaly detection training with TF–IDF vectorization.
- **Prometheus CPU Time-Series:** 10,080 CPU usage data points were generated as a time-series with realistic patterns including diurnal cycles, random spikes, gradual trends, and noise. This dataset was used for training the LSTM forecasting model with a sliding-window approach.

5.2 Data Collection Infrastructure

In addition to synthetic data, a complete data collection infrastructure was implemented to enable integration with live Jenkins and Prometheus instances:

- **Jenkins Collector** (`collectors/jenkins_collector.py`): A Python module that connects to Jenkins via its REST API using HTTP Basic Authentication. It retrieves build metadata (result, duration, timestamp), console logs, and test reports for specified jobs. Collected data is stored as structured CSVs and individual log files.
- **Prometheus Collector** (`collectors/prometheus_collector.py`): A module that queries the Prometheus HTTP API to fetch time-series metrics such as CPU utilization, memory usage, and pod replica counts. It supports both instant and range queries using PromQL and exports results in structured CSV format.
- **Real-Time Collector** (`collectors/realtime_collector.py`): A streaming collector designed for continuous, near-real-time data ingestion from both Jenkins and Prometheus sources.

A demonstration Next.js Todo application is included as the CI/CD target. It exposes a Prometheus-format metrics endpoint (`/api/metrics`) and ships with a Jenkinsfile defining a six-stage pipeline (Checkout → Install → Lint → Test → Build → Archive), enabling the framework to be exercised against a live build and monitoring environment.

5.3 Machine Learning Models

All three machine learning models were implemented as standalone training scripts with clear preprocessing, training, evaluation, and serialization stages.

5.3.1 XGBoost Failure Prediction Model

The XGBoost classifier (`training/train_failure_xgboost.py`) was trained on Jenkins build metadata to predict whether a CI/CD build will fail before execution.

- **Input Features:** duration, warnings, errors, failures, log_length, tests_failed, tests_passed, tests_total (8 features total).
- **Target Variable:** Binary classification: 1 for FAILURE, 0 for non-failure (SUCCESS/UNSTABLE).
- **Train/Test Split:** 80%/20% stratified split with random seed 42.
- **Hyperparameters:** 100 estimators, max depth 6, learning rate 0.1, subsample 0.8, colsample_bytree 0.8, with automatic class-weight balancing via `scale_pos_weight`.
- **Output:** Trained model saved as `orchestrator/models/xgboost_failure.bin`.

5.3.2 TF–IDF + Autoencoder Log Anomaly Detection Model

The anomaly detection model (`training/train_log_anomaly.py`) uses a two-stage pipeline: TF–IDF vectorization of Jenkins console logs followed by a PyTorch Autoencoder for reconstruction-based anomaly detection.

- **Text Vectorization:** TF–IDF vectorizer with 500 features, n-gram range (1, 2), sublinear TF scaling, and English stop-word removal.

- **Autoencoder Architecture (PyTorch):** Encoder — Linear(500→128) → ReLU → BatchNorm → Dropout(0.3) → Linear(128→64) → ReLU → BatchNorm → Dropout(0.3) → Linear(64→32) → ReLU (bottleneck). Decoder — Linear(32→64) → ReLU → BatchNorm → Dropout(0.3) → Linear(64→128) → ReLU → BatchNorm → Dropout(0.3) → Linear(128→500) → Sigmoid.
- **Training:** Trained only on normal (non-anomalous) logs using MSE reconstruction loss with the Adam optimizer (lr = 0.0005).
- **Anomaly Threshold:** Determined at the 95th percentile of reconstruction errors on the normal training data. A log is flagged as anomalous when its reconstruction error exceeds this threshold.
- **Output:** TF–IDF vectorizer (`tfidf.pkl`), Autoencoder weights (`autoencoder.pth`), and threshold (`anomaly_threshold.pkl`) saved to `orchestrator/models/`.

5.3.3 LSTM CPU Forecasting Model

The LSTM model (`training/train_cpu_lstm.py`) was trained on Prometheus CPU time-series data to forecast future CPU usage.

- **Data Preprocessing:** CPU values normalized to [0, 1] using a MinMaxScaler. Sliding-window sequences were created with a lookback of 60 minutes and a forecast horizon of 10 minutes, yielding 10,011 training sequences.
- **LSTM Architecture (PyTorch):** LSTM(input=1, hidden=128, batch_first) → Dropout(0.2) → LSTM(128→64) → Dropout(0.2) → Linear(64→32) → ReLU → Linear(32→1).
- **Training:** MSE loss with the Adam optimizer (lr = 0.001), batch size 32, and learning-rate reduction on plateau (factor 0.5, patience 5).
- **Output:** LSTM weights (`lstm_cpu.pth`) and the scaler (`cpu_scaler.pkl`) saved to `orchestrator/models/`.

5.4 FastAPI AIOps Orchestrator

The orchestrator (`orchestrator/main.py`) is the central inference layer that unifies all three models behind a single REST interface. It is implemented using FastAPI with an asynchronous lifespan handler that loads every model into memory once at startup, so that subsequent inference requests are served without reloading. The application is organized into modular routers, one per capability, and a set of utility modules for Kubernetes and Prometheus access.

Table 5.1: AIOps Orchestrator — REST API Endpoints

Endpoint	Method	Description
`/`	GET	Service information and available endpoints
`/health`	GET	Liveness/health check
`/predict_failure`	POST	Returns build failure probability (XGBoost)
`/detect_anomaly`	POST	Returns anomaly score and boolean flag (Autoencoder)
`/forecast_cpu`	POST	Returns predicted CPU usage from history (LSTM)
`/autoscale`	POST	Triggers the predictive autoscaling decision

Each endpoint uses Pydantic models for request and response validation. Interactive Swagger documentation is automatically served at `/docs`. A representative request and response for the failure-prediction endpoint is shown below:

```
POST /predict_failure
{
  "duration": 120, "warnings": 5, "errors": 8, "failures": 2,
  "log_length": 25000, "tests_failed": 5,
  "tests_passed": 19, "tests_total": 24
}
-> { "failure_probability": 0.974 }
```

5.5 Predictive Autoscaler

The predictive autoscaler is implemented in two parts: a `DecisionEngine` (`autoscaler/decision_engine.py`) that encapsulates the scaling policy, and a `PredictiveScaler` (`autoscaler/scaler.py`) that applies decisions to a Kubernetes deployment using the Kubernetes Python client. The decision engine fuses the three ML signals — predicted CPU, anomaly score, and failure probability — using a priority-ordered policy, summarized in Algorithm 5.2.

Table 5.2: Predictive Autoscaler — Decision Policy (Algorithm 1)

Step	Rule
1	If anomaly score $>$ anomaly threshold $\Rightarrow$ raise a non-blocking *alert*.
2	If failure probability $> 0.6 \Rightarrow$ *HOLD* (do not scale a likely-failing build).
3	Else if predicted CPU ≥ 0.7 and replicas $<$ max $\Rightarrow$ *scale up* by $+2$.
4	Else if predicted CPU ≤ 0.3 and replicas $>$ min $\Rightarrow$ *scale down* by -1.
5	Otherwise $\Rightarrow$ *maintain* current replicas.

The engine enforces operational safety constraints: a configurable replica range (default min = 1, max = 10), a cooldown period (default 300 s) that prevents oscillation by suppressing repeated scaling actions, and bounded per-action increments. The `PredictiveScaler` reads the current replica count from the cluster, invokes the decision engine, and patches the deployment's replica count only when a scaling action is warranted. If no Kubernetes cluster is reachable, the orchestrator degrades gracefully — the other three endpoints remain fully functional while `/autoscale` reports that the cluster is unavailable.

5.6 Containerization and Kubernetes Deployment

The orchestrator and autoscaler are containerized using a single Dockerfile based on the `python:3.11-slim` image. The image installs the Python dependencies, copies the application code together with the trained model artifacts, exposes port 8000, and launches the service through Uvicorn. Kubernetes manifests are provided in `kubernetes_manifests/`:

- `orchestrator-deployment.yaml` — Deployment for the AIOps orchestrator.
- `autoscaler-deployment.yaml` — Deployment for the predictive autoscaler, including the ServiceAccount/RBAC needed to patch deployment replicas.
- `demo-app-deployment.yaml` — Deployment for the demo Todo application that acts as the scaling target.
- `service.yaml` — Service exposing the orchestrator within the cluster.

A PowerShell helper (`scripts/deploy-minikube.ps1`) automates building the image into the Minikube Docker context and applying the manifests, enabling single-command local deployment.

Chapter 6

Results and Evaluation

A comprehensive evaluation suite (`evaluation/evaluate_all.py`) loads the trained models, computes standard metrics on held-out test data, and generates twelve publication-quality plots together with a consolidated metrics report (`evaluation/metrics/all_metrics.json`). This chapter reports those results, followed by the behaviour of the predictive autoscaler.

6.1 XGBoost Failure Prediction Results

The XGBoost model demonstrated excellent performance on the 20% held-out test set:

Table 6.1: XGBoost Failure Prediction — Test Set Performance

Metric	**Value**
Accuracy	99.17%
Precision	96.43%
Recall	100.00%
F1-Score	98.18%

A 5-fold stratified cross-validation was also performed to verify model stability:

Table 6.2: XGBoost 5-Fold Cross-Validation Results

Metric	**Mean ± Std**
Accuracy	99.00% ± 0.62%
F1-Score	97.83% ± 1.37%

The low cross-validation variance confirms that the model generalizes consistently across folds rather than relying on a favourable single split. The feature importance analysis revealed that `errors`, `failures`, and `tests_failed` were the most influential predictors of build failure, which is intuitive from a DevOps standpoint.

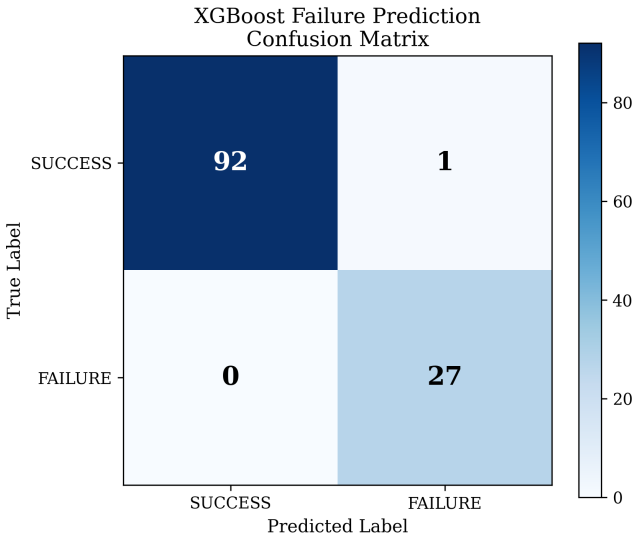


Figure 6.1: XGBoost Failure Prediction — Confusion Matrix

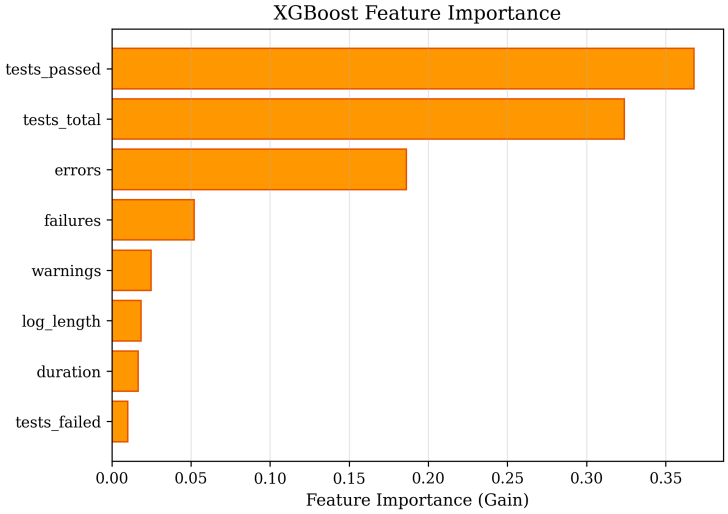


Figure 6.2: XGBoost Feature Importance Analysis

6.2 Anomaly Detection Results

The TF–IDF + Autoencoder anomaly detector was evaluated on a held-out test set drawn from the 600 console logs:

The model achieved perfect recall (100%), meaning it detected every anomalous (failure) log in the test set — an important property for an operational alerting system, where missing a genuine anomaly is far more costly than a false alarm. The reconstruction-error distribution (Figure 6.4) shows a clear separation between normal and anomalous logs around the learned

Table 6.3: Log Anomaly Detection — Test Set Performance

Metric	Value
Accuracy	96.67%
Precision	87.10%
Recall	100.00%
F1-Score	93.10%

95th-percentile threshold. It should be noted that, because the evaluation uses synthetic logs in which failure and success messages have distinctive vocabulary, the class separability is high; on noisier real-world logs the precision would be expected to decrease, and threshold calibration on live data is identified as part of the future scope.

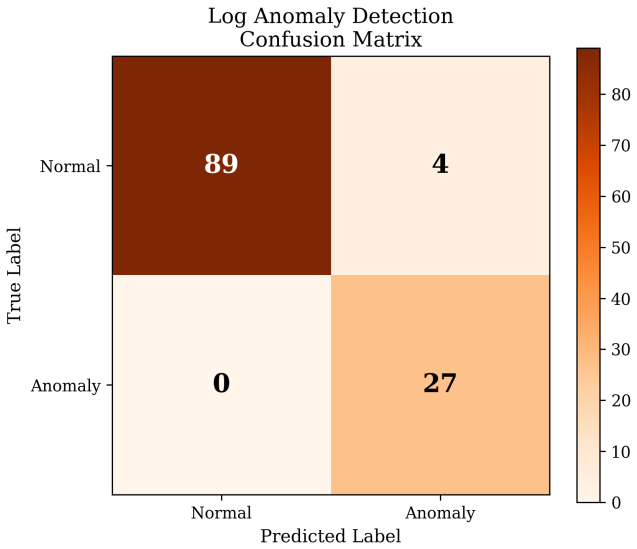


Figure 6.3: Log Anomaly Detection — Confusion Matrix

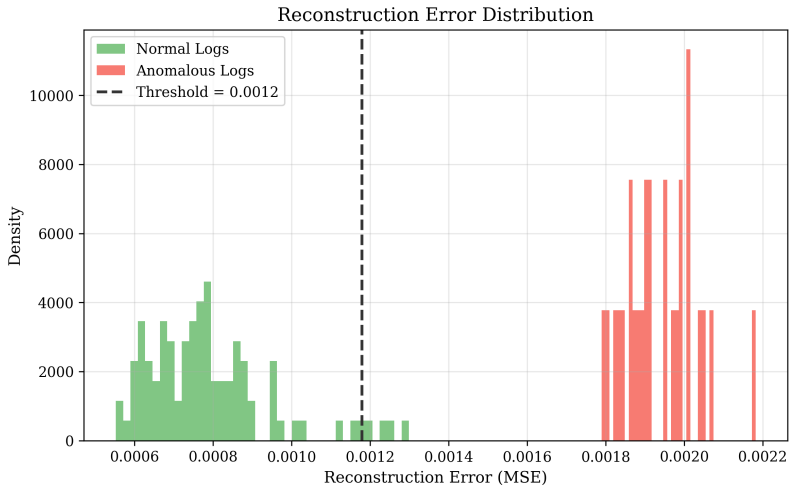


Figure 6.4: Reconstruction Error Distribution — Normal vs Anomalous Logs

6.3 LSTM CPU Forecasting Results

The LSTM model was evaluated on the temporally held-out portion of the CPU time-series:

Table 6.4: LSTM CPU Forecasting — Test Set Performance

Metric	**Value**
MSE	0.0049
MAE	0.0433
RMSE	0.0698
MAPE	9.68%
R^2	0.455

The model demonstrates strong short-term forecasting capability, with a mean absolute error of roughly 4.3% of the normalized CPU range and a MAPE under 10%. The actual-vs-predicted overlay (Figure 6.5) confirms that predictions closely track real CPU trends, and the residuals are approximately centred on zero. The moderate R^2 reflects the substantial random-noise component deliberately injected into the synthetic series; the low RMSE/MAE confirm that the model captures the underlying diurnal and trend structure that matters for proactive scaling decisions.

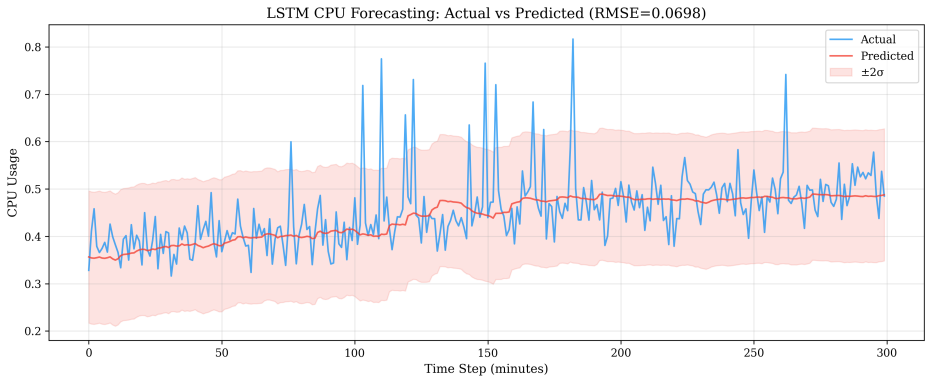


Figure 6.5: LSTM CPU Forecasting — Actual vs Predicted Values

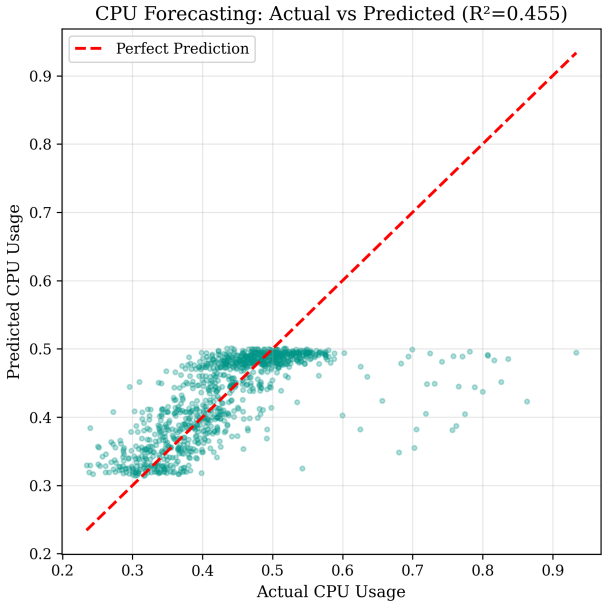


Figure 6.6: LSTM CPU Forecasting — Actual vs Predicted Scatter Plot

6.4 Combined Model Performance Summary

Table 6.5 presents a consolidated view of all three model performances.

Table 6.5: AIOps System — Combined Model Performance Summary

Module	**Algorithm**	**Key Metric**	**ROC AUC / R^2**
Failure Prediction	XGBoost	F1 = 0.982	AUC = 1.000
Anomaly Detection	TF–IDF + Autoencoder	F1 = 0.931	AUC = 1.000
CPU Forecasting	LSTM	RMSE = 0.070	R^2 = 0.455

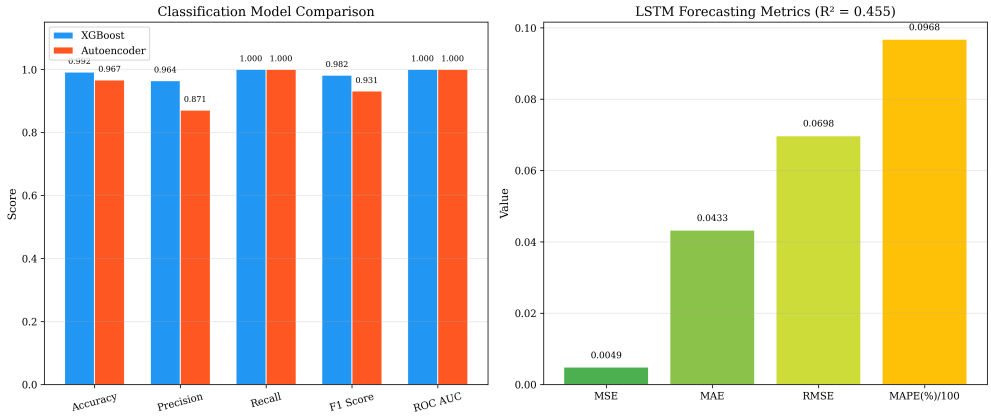


Figure 6.7: Model Comparison — Classification Metrics and Forecasting Metrics

6.5 Predictive Autoscaler Behaviour

The decision engine was exercised on a set of representative operating scenarios to verify that the fused-signal policy behaves correctly. Table 6.6 reports the actual decisions produced by the implementation for a deployment currently running 5 replicas (with the at-maximum case starting from 10). The results confirm each branch of Algorithm 5.2: high forecast CPU triggers a proactive scale-up, low forecast CPU triggers a scale-down, a high failure probability holds scaling to avoid wasting resources on a failing build, a log anomaly raises an alert while still allowing CPU-driven scaling, and the replica ceiling is respected.

Table 6.6: Predictive Autoscaler — Decisions for Representative Scenarios

Scenario	**CPU**	**Anom.**	**Fail.**	**Action**	**Replicas**
High CPU forecast	0.85	0.10	0.20	scale_up	$5 \rightarrow 7$
Low CPU forecast	0.20	0.05	0.10	scale_down	$5 \rightarrow 4$
Normal load	0.50	0.10	0.20	no_action	5
High failure prob. (hold)	0.90	0.10	0.80	no_action	5
Anomaly + high CPU	0.80	0.95	0.20	scale_up + alert	$5 \rightarrow 7$
At maximum replicas	0.95	0.10	0.20	no_action	10

Unlike the default Kubernetes HPA, which reacts only after measured CPU has already

crossed a threshold, this engine acts on a *forecast* of CPU ten minutes ahead and additionally vetoes scaling during likely-failing builds. This proactive, signal-fused behaviour is the central practical contribution of the framework.

6.6 Summary of Implementation Outcomes

The following outcomes were achieved:

- Complete synthetic data generation pipeline for Jenkins builds, console logs, and CPU time-series, plus live Jenkins/Prometheus collectors.
- XGBoost failure-prediction model: 99.17% accuracy, 98.18% F1, 99.0% mean CV accuracy.
- TF–IDF + Autoencoder anomaly detector: 96.67% accuracy, 93.10% F1, 100% recall.
- LSTM CPU forecaster: RMSE 0.070, MAE 0.043, MAPE 9.68%.
- All three models integrated into a FastAPI orchestrator that loads them at startup and serves real-time inference.
- A predictive autoscaler implemented with the Kubernetes Python client, fusing the three ML signals under a priority policy with cooldown and replica limits.
- Containerization (Docker) and Kubernetes manifests for the orchestrator, autoscaler, and demo application, with a one-command Minikube deployment script.

Chapter 7

Conclusion and Future Scope

7.1 Conclusion

This dissertation set out to replace reactive DevOps mechanisms with a proactive, data-driven AIOps framework, and it has achieved that goal end-to-end. Three machine learning models were designed, trained, and rigorously evaluated: an XGBoost classifier that predicts CI/CD build failures before execution, a TF–IDF + Autoencoder model that flags anomalous Jenkins console logs without requiring labelled anomalies, and an LSTM network that forecasts short-term Kubernetes CPU usage. On held-out test data the models attained 99.17% accuracy (failure prediction), 96.67% accuracy with perfect recall (anomaly detection), and an RMSE of 0.070 with a MAPE under 10% (CPU forecasting).

Crucially, the work goes beyond isolated models. All three were unified behind a single FastAPI orchestrator that loads them at startup and serves real-time inference, and their outputs were fused by a predictive autoscaler that makes proactive Kubernetes scaling decisions while respecting cooldown intervals and replica limits. The entire pipeline — from data ingestion through inference to autoscaling — was containerized and deployed using Docker and Kubernetes manifests. In doing so, the dissertation directly addresses the central research gap identified in the literature: the absence of a single framework that combines failure prediction, log anomaly detection, and predictive autoscaling. The system demonstrates that fusing CI/CD signals with resource forecasts enables earlier, better-informed operational decisions than any of these signals could provide in isolation.

7.2 Future Scope

While the framework is complete and functional, several directions would strengthen it for production use:

1. **Validation on real-world data:** Replacing the synthetic datasets with logs and metrics collected from production Jenkins and Prometheus instances, and recalibrating the anomaly threshold on live traffic, to confirm the metrics generalize beyond the synthetic separability of the current logs.
2. **Quantitative benchmarking against HPA:** Running controlled cluster experiments that compare the predictive autoscaler against the default Horizontal Pod Autoscaler on response time, SLA violations, and resource cost.
3. **Stronger log models:** Evaluating transformer-based log encoders (e.g., LogBERT-style models) against the TF–IDF + Autoencoder baseline where compute permits, to capture sequence-level context.

4. **Online / continual learning:** Periodically retraining the models on freshly collected data so that they adapt to drift in pipeline behaviour and workload patterns.
5. **Monitoring dashboard:** A Grafana or Streamlit dashboard surfacing live predictions, anomaly alerts, CPU forecasts, and autoscaling actions, together with Prometheus-based alerting.
6. **Closed-loop remediation:** Extending the autoscaler into a broader remediation engine that can, for example, automatically retry or quarantine builds predicted to fail.

Bibliography

[1] I. Pintye, B. Goldschmidt, P. Holicza and A. Kertesz, "Enhancing Machine Learning-Based Autoscaling for Cloud Resource Orchestration," *Journal of Grid Computing*, vol. 22, pp. 1–18, 2024.

[2] P. B. Guruge and Y. H. P. P. Priyadarshana, "Time Series Forecasting-Based Kubernetes Autoscaling Using Facebook Prophet and Long Short-Term Memory," *Frontiers in Computer Science*, vol. 5, 2025.

[3] M. A. Yahya, W. Bakhtawar and M. Usman, "Deep Learning for Multivariate Time Series Anomaly Detection: A Survey of Reconstruction-Based Methods," *Artificial Intelligence Review*, 2025.

[4] A. Hrusto, "Towards Optimization of Anomaly Detection in DevOps," *Information and Software Technology*, vol. 158, 2023.

[5] S. Das and P. Raj, "AI-Enhanced Platform Engineering: Revolutionizing CI/CD Pipelines Through Intelligent Automation," *International Journal of Computer Engineering Science and Technology*, vol. 3, pp. 1–12, 2025.

[6] Y. M. Abdelhamid, W. A. Aziz and J. N. Soliman, "Kubernetes Autoscaling with Machine Learning Based on Traffic Load Prediction," ResearchGate Preprint, May 2025.

[7] R. Mathew and S. Baswareddy, "Predictive Failure Analysis in Continuous Integration Pipelines Using Ensemble Trees," *International Journal of Advanced Computer Science*, vol. 14, pp. 221–230, 2025.

[8] H. Guo, J. Zhou, P. Luo and X. Chen, "LogBERT: Transformer-Based Log Anomaly Detection for CI/CD Systems," in *Proc. 35th Conference on Software Engineering*, 2021.

[9] P. Sharma and R. Kumar, "Auto-Scaling Techniques for Container Workloads in Kubernetes Clusters: A Review," *Journal of Cloud Computing*, vol. 12, pp. 1–15, 2025.

[10] S. Houerbi, J. da Silva Santos and A. Zaidman, "Empirical Analysis on CI/CD Pipeline Evolution: Failures and Bottlenecks," *Journal of Software Maintenance and Evolution*, vol. 36, pp. e2720, 2024.

[11] Kubernetes Documentation - Autoscaling Concepts. [Online]. Available: `https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/`

[12] Prometheus Documentation - Querying and Metrics. [Online]. Available: `https://prometheus.io/docs/`

[13] Jenkins Documentation - Pipeline and REST API. [Online]. Available: `https://www.jenkins.io/doc/`